

Documentación general

Arquitectura, módulos y cómo se conecta todo

Sistema de venta de tecnología · **Spring Boot** (API REST) + **React** (Vite) + **MySQL**

Los apartados marcados con  indican qué se puede modificar; los de , puntos delicados.

- 
1. ¿Qué es GamerStore?
 2. Las 3 piezas que se ejecutan
 3. El viaje de una petición (memoriza esto)
 4. Módulos del sistema
 - Tienda pública
 - Panel ERP
 5. Recorrido completo de una venta (el flujo estrella)
 6. Seguridad en 1 minuto
 7. Integraciones externas
 8. Base de datos
 9. Dónde tocar cada cosa (mapa rápido)
 10. Estado del proyecto

Este documento es el **punto de entrada**. Explica qué es el sistema, cómo está armado y cómo se conecta todo. Para el detalle archivo por archivo:

Documento	Qué contiene
DOC-BACKEND.md	Cada archivo Java: entidades, repositorios, servicios, controladores, seguridad, boleta. Con guía de "qué puedo modificar".
DOC-FRONTEND.md	Cada archivo React: capa de API, contextos, hooks, componentes, páginas. Con guía de "qué puedo modificar".
COMO-EJECUTAR.md	Pasos para levantar el proyecto en otra PC.

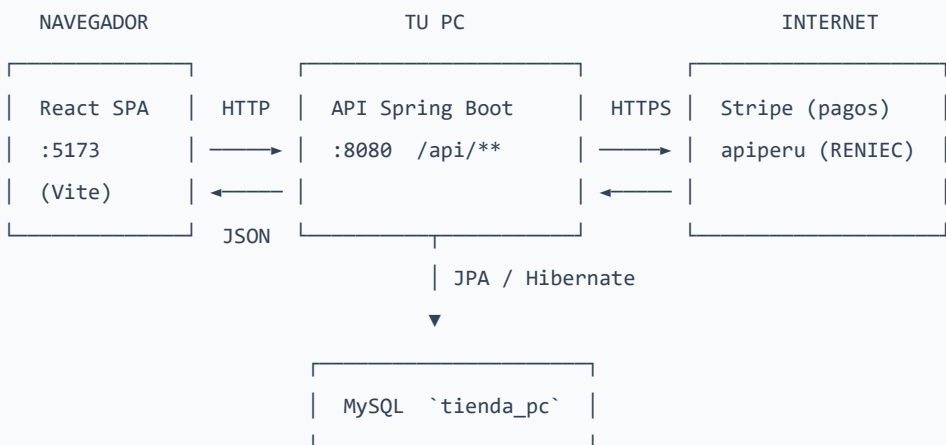
1. ¿Qué es GamerStore?

Un sistema de venta de productos de tecnología con **dos caras**:

- **Tienda pública** (/): el cliente navega el catálogo, arma su carrito y **paga en línea** (tarjeta con Stripe o Yape). Al terminar recibe su **boleta**.
- **Panel administrativo / ERP** (/admin): el negocio gestiona catálogo, clientes, pedidos, usuarios, y consulta el **registro de ventas** con IGV.

Toda venta pasa por el ecommerce. No existe venta por WhatsApp (eso era el sistema anterior); WhatsApp quedó solo como canal de contacto.

2. Las 3 piezas que se ejecutan



Pieza	Tecnología	Puerto	Se levanta con
Frontend	React 18 + Vite 5	5173	<code>npm run dev</code> (en <code>frontend/</code>)
Backend	Spring Boot 3.5.6 (Java 17)	8080	<code>mvnw.cmd spring-boot:run</code>
Base de datos	MySQL/MariaDB (XAMPP)	3306	XAMPP → Start MySQL

En desarrollo, **Vite hace de proxy**: cuando el navegador pide `/api/...` o `/images/...`, Vite lo reenvía al backend en `:8080`. Por eso el front nunca escribe `http://localhost:8080` en el código. Se configura en `frontend/vite.config.js`.

3. El viaje de una petición (memoriza esto)

Todo el sistema funciona igual. Si entiendes este recorrido, entiendes cualquier botón:

IDA: Pantalla React → endpoints.js → client.js (+token) → Controller → Service → Repository → BD
VUELTA: BD → Repository → Service → Mapper (Entidad→DTO) → Controller (JSON) → client.js → setState
→ pantalla

Capa	Dónde vive	Responsabilidad
Pantalla	frontend/src/pages/...	Muestra datos y captura acciones del usuario
endpoints.js	frontend/src/api/	Sabe la URL de cada operación
client.js	frontend/src/api/	Hace el <code>fetch</code> , agrega el token, refresca la sesión, normaliza errores
Controller	.../controller/	Recibe el HTTP, valida el formato de entrada
Service	.../service/	Reglas de negocio (validaciones, cálculos, decisiones)
Repository	.../repository/	Habla con la base de datos
Mapper / DTO	.../mapper/ , .../dto/	Convierte entidades a objetos "de salida" (JSON)

Regla de oro: la lógica de negocio va en el **Service**. El Controller solo recibe y responde; el Repository solo consulta.

4. Módulos del sistema

Tienda pública

Módulo	Ruta	Qué hace
Catálogo	/productos	Lista y filtra productos por categoría o búsqueda
Detalle	/productos/:id	Ficha del producto + agregar al carrito
Carrito	/carrito	Ítems, cantidades (topadas al stock) y total
Checkout	/checkout	Asistente de 4 pasos: identificación → datos → pago → confirmar

Panel ERP

Módulo	Ruta	Qué hace
Dashboard	/admin	KPIs, gráficos, stock bajo, más vendidos
Productos	/admin/productos	CRUD + subida de imagen + nombre único
Categorías	/admin/categorias	CRUD + bloqueo si tiene productos
Clientes	/admin/clientes	CRUD + autocompletado por DNI (RENIEC)
Pedidos	/admin/pedidos	Operación: estados, cobro y detalle con pago + boleta
Ventas	/admin/ventas	Registro de ventas: boletas emitidas, IGV y totales
Usuarios	/admin/usuarios	CRUD de accesos + protección del último admin

¿Por qué Pedidos y Ventas están separados? Un **pedido** es la operación: cambia de estado (PAGADO → ENVIADO → ENTREGADO). Una **boleta** es el documento contable: se emite una vez y **no se modifica nunca** (si hay error se anula con nota de crédito). Son la misma transacción vista desde la operación y desde la contabilidad. El **pago** no es un módulo: es un dato del pedido, por eso vive dentro de su detalle.

5. Recorrido completo de una venta (el flujo estrella)

1. Cliente agrega productos → CarritoContext (localStorage)
2. Va al checkout y se identifica → POST /api/checkout/cliente (o compra como invitado)
 - └ escribe su DNI → GET /api/reniec/{dni} (autocompleta con RENIEC)
3. Elige método de pago
 - └ Tarjeta → Stripe.js tokeniza en el navegador → paymentMethodId
 - └ Yape → muestra QR + N° de operación (solo si el total ≤ S/ 500)
4. Confirma → POST /api/checkout
 - └ CheckoutService (transaccional):
 - | └ crea o reutiliza el Cliente (por DNI)
 - | └ PedidoService.crear() → valida y DESCUENTA stock, calcula el total
 - | └ PagoService → cobra (Stripe real / Yape / simulado)
 - | └ si aprueba → pedido PAGADO + ComprobanteService.emitir() → BOLETA
 - └ si el pago se RECHAZA → 402 y se revierte TODO (ni pedido, ni stock, ni pago)
5. Confirmación en pantalla + descarga de boleta → GET /api/checkout/boleta/{codigo}?dni=
6. En el ERP aparece en Pedidos y en Ventas

Puntos clave para defender ante el profesor: - El **total y el stock se calculan en el backend**: no se confía en lo que manda el navegador (no se puede manipular el precio). - La compra es **transaccional**: o pasa todo, o no pasa nada. - La **tarjeta nunca toca nuestro servidor** (Stripe la tokeniza en el navegador) → estándar PCI.

6. Seguridad en 1 minuto

Mecanismo	Cómo funciona
Login	Usuario/clave → se valida con BCrypt → devuelve access token (JWT, 5 min) + refresh token (BD, 7 días)
Autorización	Cada petición lleva <code>Authorization: Bearer ...</code> . <code>JwtAuthenticationFilter</code> valida y <code>SecurityConfig</code> exige rol ADMIN en <code>/api/admin/**</code>
Refresco silencioso	Si el access vence (401), el front pide uno nuevo y reintenta solo ; el usuario no se entera
Logout real	El refresh token se revoca en la BD (por eso se guarda ahí)
Anti fuerza bruta	3 intentos fallidos → bloqueo de 30 s (aunque la clave sea correcta)
Contraseñas	Siempre hasheadas con BCrypt , nunca en texto plano

7. Integraciones externas

Servicio	Para qué	Si falla...
Stripe (modo prueba)	Cobro real con tarjeta (tokenización + cargo)	Se puede desactivar (<code>app.stripe.enabled=false</code>) y usa la pasarela simulada
apiperu.dev	Datos reales por DNI (RENIEC)	<i>Best-effort</i> : si no responde, el formulario sigue funcionando manualmente
Yape	Cobro por QR (el cliente yapea y se registra el N° de operación)	Solo para montos $\leq$ S/ 500

⚠ **Las claves privadas NO están en el repositorio.** Viven en `src/main/resources/application-local.properties` , que está en `.gitignore` (GitHub bloquea los push con secretos). Hay una plantilla: `application-local.properties.ejemplo` .

8. Base de datos

categoria —1:N→ producto ←N:1— pedido_item —N:1→ pedido —N:1→ cliente
|
├1:1→ pago
└1:1→ comprobante (boleta)

usuario —1:N→ refresh_token

Tabla	Guarda	Dato clave
producto	Catálogo	nombre único, imagen = ruta (no el archivo)
categoria	Familias de productos	nombre único
cliente	Compradores	dni único
pedido	La operación de venta	estado , total
pedido_item	Línea del pedido	Guarda el precio del momento
pago	Transacción	referencia (Stripe pi_... o N° de operación Yape)
comprobante	Boleta	serie + numero correlativo, subtotal / igv / total
usuario	Accesos al ERP	username / email únicos, clave BCrypt
refresh_token	Sesiones activas	Permite revocar

Las tablas **las crea Hibernate solo** a partir de las clases `@Entity` . No se escribe SQL a mano.

9. Dónde tocar cada cosa (mapa rápido)

Quiero cambiar...	Voy a...
Un valor de negocio (IGV, tope de Yape, intentos de login, duración del token)	<code>src/main/resources/application.properties</code>
Una clave privada (Stripe, apiperu)	<code>application-local.properties</code> (no se sube)
Una regla de negocio (validaciones, cálculos)	El Service correspondiente
Una pantalla, texto o color	<code>frontend/src/pages/...</code> o <code>frontend/src/index.css</code>
El diseño de la boleta	<code>src/main/resources/templates/boleta.html</code> (¡es HTML/CSS!)
Los productos iniciales	<code>config/DataSeeder.java</code>
Qué rutas son públicas	<code>config/security/SecurityConfig.java</code>

El detalle completo, archivo por archivo, está en `DOC-BACKEND.md` y `DOC-FRONTEND.md`.

10. Estado del proyecto

-  Pruebas de integración: **8/8** (`mvnw test`)
-  Build del frontend sin errores (`npm run build`)
-  Pagos verificados contra la API real de Stripe (modo prueba)
-  La boleta lleva la leyenda "**DOCUMENTO DE DEMOSTRACIÓN — SIN VALIDEZ TRIBUTARIA**": para emitir comprobantes fiscales reales hacen falta RUC y certificado digital, y transmitir el XML a SUNAT. La estructura del documento ya es la correcta.